



Relatório Técnico-Operacional

Tutorial 2: Ingestão de Dados com DLTHub (dlt) + Python na Nuvem

Atividade: Execução De Pipeline Programático Em Nuvem Com DLT, RDS PostgreSQL, PokéAPI e Amazon S3

Disciplina: Ecossistema de Big Data

Professor: Leandro Mendes Ferreira

Grupo A

Nome	Matrícula USP
Breno Tostes Garcia	18105650
Gerson Chadi Junior	4360987
Isabella Abreu Comelli	18106391
Fernando Luiz Gomes	18105793
Caetano Sales Carvalho	18105692

São Paulo, 28 de junho de 2026

1. INTRODUÇÃO

A presente atividade tem a finalidade de executar, em ambiente de nuvem, um pipeline de ingestão de dados utilizando a biblioteca DLTHub/dlt em Python. O exercício corresponde ao Tutorial 3 (Ingestão de Dados com DLTHub (dlt) + Python na Nuvem), no qual o processamento ocorre em uma instância Amazon EC2, a origem relacional encontra-se em um banco PostgreSQL hospedado no Amazon RDS e o destino final dos dados é um bucket Amazon S3. Adicionalmente, foi realizada ingestão de uma fonte externa pública, a PokéAPI, com armazenamento dos resultados também no S3 em formato Parquet.

A atividade complementa os exercícios anteriores. O Tutorial 1 (Infraestrutura de Ingestão na Nuvem com Terraform) estruturou a infraestrutura de nuvem com Terraform, enquanto o Tutorial 2 (Ingestão com Meltano na Nuvem) validou a ingestão com Meltano. O Tutorial 3, por sua vez, reproduziu a ingestão por meio de scripts Python, permitindo demonstrar uma abordagem programática para construção de pipelines de dados.

2. OBJETIVO DA ATIVIDADE

O objetivo geral foi implementar e validar um pipeline de ingestão em nuvem com dlt, com leitura de dados do RDS PostgreSQL e da PokéAPI e gravação de arquivos Parquet no Amazon S3.

- confirmar a disponibilidade do ambiente EC2, RDS e S3 criado anteriormente;
- preparar um ambiente virtual Python na EC2;
- instalar o dlt com os extras necessários para leitura SQL, escrita em filesystem/S3 e serialização em Parquet;
- configurar variáveis de ambiente para endpoint do RDS, senha do banco e bucket S3;
- executar ou validar a execução de pipelines para as fontes PostgreSQL/RDS e PokéAPI;
- verificar a publicação dos arquivos Parquet e metadados operacionais no S3;
- registrar problemas encontrados e soluções adotadas para fins de reprodutibilidade.

3. AMBIENTE COMPUTACIONAL UTILIZADO

A Tabela 3-1 apresenta os principais componentes computacionais empregados na execução do Tutorial 3, indicando a função de cada recurso no desenvolvimento do pipeline de ingestão. Observa-se que o ambiente local Windows com PowerShell foi utilizado para recuperação de parâmetros da infraestrutura e acesso remoto à EC2, enquanto o AWS Academy Learner Lab forneceu o ambiente de nuvem com credenciais temporárias. A instância Amazon EC2 atuou como nó de execução dos comandos Linux e dos scripts Python, o Amazon RDS PostgreSQL funcionou como fonte relacional dos dados, e o Amazon S3 foi utilizado como camada de armazenamento dos arquivos Parquet. A tabela também registra o uso do Python 3.11 com ambiente virtual e da biblioteca DLTHub/dlt 1.28.1, evidenciando os elementos necessários para a execução programática do pipeline.

Tabela 3-1: Componentes computacionais utilizados na execução do pipeline de ingestão com DLTHub/dlt em ambiente AWS

Componente	Uso na atividade
Windows + PowerShell	Terminal local utilizado para recuperar valores do Terraform, validar o S3 e acessar a EC2 por SSH.
AWS Academy Learner Lab	Ambiente de nuvem utilizado no laboratório, com credenciais temporárias.
Amazon EC2	Ambiente de execução dos comandos Linux, instalação do dlt e execução dos scripts Python.
Amazon RDS PostgreSQL	Banco de dados relacional de origem, contendo a tabela vendas previamente populada.
Amazon S3	Camada de armazenamento de destino, utilizada como data lake para os arquivos Parquet.
Python 3.11 + venv	Ambiente isolado para instalação das dependências do Tutorial 3.
DLTHub/dlt 1.28.1	Biblioteca utilizada para carregar dados do PostgreSQL e da PokéAPI para o S3.

4. ARQUITETURA LÓGICA DA SOLUÇÃO

A arquitetura implementada utiliza a EC2 como nó de execução. A partir dela, o dlt acessa o RDS PostgreSQL pela rede privada da AWS e, simultaneamente, consulta a PokéAPI pela internet. Os dados são serializados em Parquet e gravados no Amazon S3. O acesso ao S3 ocorre por meio do perfil IAM associado à instância EC2, sem necessidade de configurar chaves de acesso AWS diretamente no código.

```
RDS PostgreSQL / tabela vendas
-> dlt + Python na EC2
-> arquivos Parquet
-> S3: s3://650379606721-ingestao-lab/postgres/vendas/
```

```
PokéAPI / recurso pokemon
-> dlt + Python na EC2
-> arquivos Parquet
-> S3: s3://650379606721-ingestao-lab/pokeapi/pokemon/
```

5. ETAPAS EXECUTADAS

5.1.Preparação Das Variáveis Do Laboratório

Foram configuradas variáveis de ambiente para evitar a digitação manual e repetitiva do endpoint do RDS, da senha do banco e do caminho do bucket S3. Essas variáveis foram utilizadas pelos comandos de validação e pelos scripts Python.

```
source ~/dlt-vars.sh

echo $RDS_ENDPOINT
echo $DB_PASSWORD
echo $BUCKET
echo $DESTINATION__FILESYSTEM__BUCKET_URL
```

5.2.Validação Da Origem Relacional No RDS

Antes da execução do dlt, confirmou-se que o banco PostgreSQL estava populado. Essa etapa foi importante porque o pipeline PostgreSQL dependia da existência de registros na tabela vendas.

```
export PGPASSWORD=ecommerce123

psql -h $RDS_ENDPOINT -U ecommerce -d ecommerce -c "SELECT 'clientes' AS tabela, count(*) FROM
clientes UNION ALL SELECT 'produtos', count(*) FROM produtos UNION ALL SELECT 'vendas', count(*)
FROM vendas;"
```

O resultado esperado e utilizado como evidência operacional foi:

```
tabela | count
-----+-----
clientes | 20
produtos | 15
vendas   | 200
```

5.3. Criação Do Diretório Do Projeto Dlt

Foi criada uma pasta específica para o Tutorial 3, separando a execução do dlt da atividade anterior com Meltano.

```
cd ~
rm -rf projeto_dlt_aws
mkdir projeto_dlt_aws
cd projeto_dlt_aws
```

5.4.Criação E Ativação Do Ambiente Virtual Python

O ambiente virtual foi utilizado para isolar as dependências Python do Tutorial 3. Esse procedimento reduz conflitos entre bibliotecas e torna a execução mais controlável.

```
python3.11 -m venv .venv
source .venv/bin/activate
```

5.5. Instalação Do DLT E Dependências

O DLT foi instalado com os extras necessários para acesso a bancos SQL, uso de filesystem/S3 e gravação em Parquet. Também foi instalado o driver psycopg2-binary para comunicação com o PostgreSQL.

```
pip install --upgrade pip
pip install "dlt[filesystem,sql_database,parquet]" psycopg2-binary

dlt --version
```

A instalação foi validada pela saída:

```
dlt 1.28.1
```

5.6. Script Para Ingestão Postgresql/RDS

O pipeline PostgreSQL foi definido com a finalidade de extrair a tabela vendas do banco ecommerce e gravá-la no S3 no conjunto de dados postgres. O código abaixo representa o script necessário para reproduzir a carga.

```
cat > pipeline_postgres.py <<EOF
import os

import dlt
from dlt.sources.sql_database import sql_table

rds_endpoint = os.environ["RDS_ENDPOINT"]
db_password = os.environ["DB_PASSWORD"]

pipe = dlt.pipeline(
    pipeline_name="ingestao_postgres",
    destination="filesystem",
    dataset_name="postgres",
)

fonte = sql_table(
    credentials=f"postgresql://ecommerce:{db_password}@{rds_endpoint}:5432/ecommerce",
    table="vendas",
)

info = pipe.run(fonte, loader_file_format="parquet")
print(info)
EOF
```

```
python pipeline_postgres.py
```

5.7. Script Para Ingestão Da Pokéapi

O pipeline da PokéAPI foi definido com paginação por offset, utilizando o recurso pokemon. O resultado foi gravado no dataset pokeapi em formato Parquet.

```
cat > pipeline_api.py <<'EOF'
import dlt
from dlt.sources.rest_api import rest_api_source

fonte = rest_api_source({
    "client": {
        "base_url": "https://pokeapi.co/api/v2/"
    },
    "resources": [
        {
            "name": "pokemon",
            "endpoint": {
                "path": "pokemon",
                "data_selector": "results",
                "paginator": {
                    "type": "offset",
                    "limit": 100,
                    "offset": 0,
                    "limit_param": "limit",
                    "offset_param": "offset",
                    "total_path": "count",
                    "maximum_offset": 200
                }
            }
        }
    ]
})

pipe = dlt.pipeline(
    pipeline_name="ingestao_pokeapi",
    destination="filesystem",
    dataset_name="pokeapi",
)

info = pipe.run(fonte, loader_file_format="parquet")
print(info)
EOF
```

```
python pipeline_api.py
```

6. VALIDAÇÃO DOS RESULTADOS NO AMAZON S3

A validação foi realizada por meio do AWS CLI, listando recursivamente os objetos gravados no bucket S3. O resultado confirmou a existência de arquivos Parquet nas pastas postgres/vendas e pokeapi/pokemon, bem como metadados internos do DLT.

```
aws s3 ls "s3://$BUCKET/postgres/" --recursive
aws s3 ls "s3://$BUCKET/pokeapi/" --recursive
aws s3 ls "s3://$BUCKET/" --recursive
```

A Tabela 6-1 apresenta o resultado da consulta de validação realizada no banco PostgreSQL hospedado no Amazon RDS antes da execução dos pipelines com dlt. A verificação indicou a existência de 20 registros na tabela `clientes`, 15 registros na tabela `produtos` e 200 registros na tabela `vendas`. Esse procedimento teve papel fundamental na confirmação da disponibilidade da fonte relacional, especialmente porque o pipeline PostgreSQL dependia da existência de dados previamente carregados na tabela `vendas`. Assim, a tabela demonstra que o banco `ecommerce` estava populado e apto a servir como origem de dados para o processo de ingestão em nuvem.

Tabela 6-1: Objetos principais identificados no Amazon S3 após a execução dos pipelines com DLTHub/dlt.

Origem	Objeto principal observado no S3	Interpretação
RDS PostgreSQL	postgres/vendas/1782528565.9891613.1126ae1607.parquet	Confirma a ingestão da tabela vendas para o S3 em formato Parquet.
PokéAPI	pokeapi/pokemon/1782528600.4203684.5ba2ea3316.parquet	Confirma a ingestão do recurso pokemon para o S3 em formato Parquet.
Metadados dlt	postgres/_dlt_* e pokeapi/_dlt_*	Arquivos de controle da execução, versão e estado dos pipelines.

7. PROBLEMAS ENCONTRADOS E SOLUÇÕES ADOTADAS

A Tabela 7-1 consolida os principais problemas operacionais identificados ao longo da execução do Tutorial 3, relacionando cada ocorrência à respectiva causa provável e ao tratamento adotado. Os registros evidenciam dificuldades associadas à reprodutibilidade dos scripts, à necessidade de execução dentro da instância EC2, ao volume de dependências instaladas pelo dlt e à presença de arquivos

auxiliares gerados automaticamente no S3. A tabela demonstra que os problemas observados não comprometeram a validação final do exercício, uma vez que os arquivos Parquet foram localizados no bucket S3. Contudo, reforça a necessidade de manter os scripts `pipeline_postgres.py` e `pipeline_api.py` salvos, versionados ou anexados ao relatório, de modo a garantir rastreabilidade, auditoria e repetição do experimento.

Tabela 7-1: Problemas operacionais identificados durante a execução do Tutorial 3, causas prováveis e tratamentos adotados

Problema observado	Causa provável	Tratamento adotado
Arquivos <code>pipeline_postgres.py</code> e <code>pipeline_api.py</code> ausentes na pasta <code>projeto_dlt_aws</code>	Os scripts não estavam salvos no diretório corrente no momento da verificação posterior.	Recriação dos scripts no diretório do projeto para garantir reprodutibilidade, apesar de os arquivos Parquet já constarem no S3.
Necessidade de execução dentro da EC2	O RDS é privado e acessível pela VPC; portanto, a execução local no Windows não é adequada para a ingestão.	Os comandos Python, <code>psql</code> e validações de rede foram executados dentro da EC2.
Dependências Python numerosas	O <code>dlt</code> com extras <code>filesystem</code> , <code>sql_database</code> e <code>parquet</code> instala bibliotecas como <code>s3fs</code> , <code>pyarrow</code> , <code>SQLAlchemy</code> e <code>psycopg2-binary</code> .	Foi utilizado ambiente virtual <code>.venv</code> para isolar as dependências e evitar conflitos.
Arquivos auxiliares <code>_dlt_*</code> no S3	Comportamento normal do <code>dlt</code> , que grava metadados de cargas, estado e versão.	Os arquivos foram interpretados como metadados operacionais, não como erro de execução.

8. EVIDÊNCIAS CONSOLIDADAS

A Tabela 8-1 reúne as principais evidências utilizadas para comprovar a execução e a validação do pipeline programático desenvolvido com DLTHub/dlt. A tabela registra a validação do banco PostgreSQL, a instalação bem-sucedida do `dlt` na versão 1.28.1, a geração dos arquivos Parquet referentes às fontes PostgreSQL e PokéAPI, a presença dos metadados operacionais do `dlt` no Amazon S3 e a permanência dos objetos produzidos no tutorial anterior com Meltano. Ao associar cada evidência ao resultado observado, a tabela funciona como síntese comprobatória da atividade e demonstra que o pipeline cumpriu seu objetivo de extrair dados de fontes heterogêneas, convertê-los para formato Parquet e disponibilizá-los em uma camada de armazenamento em nuvem.

Tabela 8-1: Síntese das evidências de execução e validação do pipeline de ingestão com DLTHub/dlt na AWS

Evidência	Resultado observado
Validação do banco PostgreSQL	clientes = 20; produtos = 15; vendas = 200
Instalação do dlt	dlt 1.28.1
Arquivo Parquet do PostgreSQL	postgres/vendas/1782528565.9891613.1126ae1607.parquet
Arquivo Parquet da PokéAPI	pokeapi/pokemon/1782528600.4203684.5ba2ea3316.parquet
Metadados dlt	Objetos <code>_dlt_loads</code> , <code>_dlt_pipeline_state</code> e <code>_dlt_version</code> presentes no S3
Comparação com Tutorial 2	Permaneceram no bucket os objetos meltano/vendas e meltano/pokemon gerados anteriormente

9. INTERPRETAÇÃO TÉCNICA

A execução confirmou a viabilidade de um pipeline programático em nuvem, baseado em Python e dlt, para ingestão de dados de fontes heterogêneas. A tabela vendas, localizada em um banco PostgreSQL privado no RDS, foi carregada no S3 em formato Parquet. De forma complementar, dados da PokéAPI foram coletados via HTTP e persistidos no mesmo bucket. A organização dos dados em prefixos distintos facilita a rastreabilidade das fontes e a posterior exploração em etapas de lakehouse, catálogo, consulta SQL ou processamento distribuído.

O exercício também evidenciou a importância da reprodutibilidade. Embora os objetos Parquet tenham sido validados no S3, a ausência posterior dos scripts no diretório do projeto indica a necessidade de manter os arquivos `pipeline_postgres.py` e `pipeline_api.py` versionados ou anexados ao relatório. Essa prática é recomendável para garantir auditoria, repetição do experimento e rastreamento do código efetivamente utilizado.

10. CONSIDERAÇÕES FINAIS

O Tutorial 3 foi concluído de forma satisfatória, uma vez que o ambiente Python foi preparado na EC2, o dlt foi instalado com sucesso e os objetos resultantes das cargas foram localizados no Amazon S3. Os arquivos Parquet em postgres/vendas e pokeapi/pokemon constituem a principal evidência de conclusão do exercício.

Os registros dos scripts utilizados para a consolidação das evidências dos comandos de validação e da indicação do procedimento seguro para encerramento da infraestrutura encontram-se dispostos nos Apêndices a seguir. A destruição dos recursos com Terraform ocorreu somente após a conferência final dos relatórios e dos resultados obtidos, a fim de evitar consumo desnecessário de recursos no AWS Academy Learner Lab sem comprometer a rastreabilidade da atividade.

APÊNDICE A – REGISTRO DOS SCRIPTS UTILIZADOS

Este apêndice tem o objetivo de evidenciar os scripts utilizados nesta atividade. O registro do código permite a auditoria da execução, a repetição do experimento e a identificação precisa da lógica adotada nos pipelines de ingestão com DLTHub/dlt.

A.1. Script Pipeline_Postgres.Py

O script a seguir realiza a leitura da tabela `vendas` no banco PostgreSQL hospedado no Amazon RDS e grava o resultado no Amazon S3, em formato Parquet, no conjunto de dados Postgres.

```
import os

import dlt
from dlt.sources.sql_database import sql_table

rds_endpoint = os.environ["RDS_ENDPOINT"]
db_password = os.environ["DB_PASSWORD"]

pipe = dlt.pipeline(
    pipeline_name="ingestao_postgres",
    destination="filesystem",
    dataset_name="postgres",
)

fonte = sql_table(
    credentials=f"postgres://{ecommerce:{db_password}@{rds_endpoint}:5432/ecommerce",
    table="vendas",
)
```

```
info = pipe.run(fonte, loader_file_format="parquet")
print(info)
```

A.2. Script Pipeline_Api.Py

O script a seguir realiza a ingestão do recurso pokemon da PokéAPI, com paginação por offset, e grava o resultado no Amazon S3, em formato Parquet, no conjunto de dados pokeapi.

```
import dlt
from dlt.sources.rest_api import rest_api_source

fonte = rest_api_source({
    "client": {
        "base_url": "https://pokeapi.co/api/v2/"
    },
    "resources": [
        {
            "name": "pokemon",
            "endpoint": {
                "path": "pokemon",
                "data_selector": "results",
                "paginator": {
                    "type": "offset",
                    "limit": 100,
                    "offset": 0,
                    "limit_param": "limit",
                    "offset_param": "offset",
                    "total_path": "count",
                    "maximum_offset": 200
                }
            }
        }
    ]
})

pipe = dlt.pipeline(
    pipeline_name="ingestao_pokeapi",
    destination="filesystem",
    dataset_name="pokeapi",
)

info = pipe.run(fonte, loader_file_format="parquet")
print(info)
```

APÊNDICE B – EVIDÊNCIAS DOS COMANDOS DE VALIDAÇÃO

Este apêndice consolida os comandos e resultados que devem ser preservados como evidências da execução. O objetivo é assegurar que a validação não dependa apenas da descrição textual, mas também dos pontos de controle efetivamente observados durante a atividade.

A Tabela 0-1 consolida os principais pontos de controle utilizados para comprovar a execução e a validação do pipeline desenvolvido com DLTHub/dlt. A tabela associa cada etapa crítica da atividade ao respectivo comando ou evidência operacional, indicando também o resultado esperado ou observado. Foram contempladas a validação do banco PostgreSQL/RDS, a confirmação da instalação do dlt, a execução dos pipelines de ingestão da fonte relacional e da PokéAPI, a verificação dos arquivos Parquet publicados no Amazon S3 e a identificação dos metadados operacionais gerados automaticamente pela biblioteca dlt. Dessa forma, a tabela contribui para a reprodutibilidade do exercício, pois organiza os comandos essenciais que permitem auditar a execução, confirmar os resultados obtidos e repetir o procedimento em ambiente equivalente.

Tabela 0-1: Comandos de validação preservados como evidência operacional

Ponto de controle	Comando ou evidência	Resultado esperado ou observado
Banco PostgreSQL/RDS	psql -h \$RDS_ENDPOINT -U ecommerce -d ecommerce -c "SELECT ..."	clientes = 20; produtos = 15; vendas = 200
Instalação do dlt	dlt --version	dlt 1.28.1
Pipeline PostgreSQL	python pipeline_postgres.py	Arquivo Parquet gerado em postgres/vendas/
Pipeline PokéAPI	python pipeline_api.py	Arquivo Parquet gerado em pokeapi/pokemon/
Publicação no S3	aws s3 ls "s3://\$BUCKET/" --recursive	Objetos Parquet e metadados _dlt_* presentes no bucket

A preservação dessas evidências é relevante para demonstrar que a origem relacional estava disponível, que a biblioteca dlt foi instalada corretamente, que os pipelines foram executados e que os artefatos finais foram efetivamente publicados no Amazon S3.

APÊNDICE C – ENCERRAMENTO CONTROLADO DA INFRAESTRUTURA

Este apêndice atende à recomendação de indicar o procedimento de destruição da infraestrutura ao término das atividades dependentes. A execução do comando terraform destroy deve ocorrer apenas após a conclusão dos relatórios, a conferência dos resultados e a confirmação de que não há tutoriais subsequentes dependentes da EC2, do RDS ou do S3.

```
# Executar no PowerShell local, na pasta do Terraform do Tutorial 1
cd "D:\Users\Breno Tostes\OneDrive\Pastas\Academico\Especializacao\Engenharia-Dados-Big-Data\C2\23_ecossistema_big_data\Trabalho\EDB-006-tutoriais-main\tutoriais\ingestao\1-infraestrutura\aws\terraform"

# Conferir os recursos registrados no estado antes da destruição
terraform state list

# Destruir a infraestrutura somente após a conclusão das atividades e validações
terraform destroy
```

Como boa prática, recomenda-se salvar previamente as saídas de validação, os scripts Python e os arquivos finais do relatório antes da destruição da infraestrutura, pois a remoção dos recursos impede novas consultas diretas ao RDS, à EC2 e ao bucket S3 utilizados no laboratório.